

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

BSc Data Science & AI - Semester 2

COMPREHENSIVE REVISION NOTE 3

Propositional Logic & Inference

TOPICS COVERED IN THIS REVISION NOTE

1. Introduction to Logic in AI
2. Propositional Logic - Syntax and Semantics
3. Atomic and Complex Sentences
4. Logical Connectives (NOT, AND, OR, IMPLIES, BICONDITIONAL)
5. Truth Tables and Evaluation
6. Operator Precedence in Logic
7. Logical Equivalences and Laws
8. Inference and Entailment
9. Sound and Complete Inference
10. Inference Rules (Modus Ponens, Resolution, etc.)
11. Introduction to First-Order Logic

SECTION 1: INTRODUCTION TO LOGIC IN AI

1.1 Why Logic Matters in AI

Logic provides a formal framework for representing knowledge and reasoning about it. In AI, logic serves as the foundation for knowledge-based agents that need to store facts about the world and derive new conclusions from those facts. Unlike search algorithms that find paths through state spaces, logic allows agents to represent complex relationships, make inferences, and reason about abstract concepts.

The key advantage of logic-based approaches is that they are **declarative** - we specify WHAT we know, and the inference engine figures out HOW to derive conclusions. This separation of knowledge from reasoning makes systems more modular, easier to understand, and easier to modify.

Logic in AI operates in a **deterministic, rule-based** framework. This means we are not yet dealing with uncertainty or probability - every statement is either TRUE or FALSE, with no middle ground. Later, we will see how probabilistic reasoning extends these ideas to handle uncertainty.

■ KEY PURPOSES OF LOGIC IN AI

- 1. Knowledge Representation:** Logic provides precise languages for expressing facts about the world, relationships between objects, and rules that govern behavior.
- 2. Reasoning:** Logic provides inference rules that allow us to derive new knowledge from existing knowledge in a sound (correct) and sometimes complete (all conclusions derivable) manner.
- 3. Communication:** Logic provides an unambiguous way to communicate knowledge between humans and machines, or between different AI systems.
- 4. Verification:** Logic allows us to prove that certain conclusions follow from certain premises, which is essential for safety-critical AI applications.

1.2 Syntax vs. Semantics

Aspect	Syntax	Semantics
Definition	Rules for forming valid sentences	Rules for determining meaning/truth
Focus	Structure and form	Meaning and interpretation
Question	Is this sentence well-formed?	Is this sentence true or false?
Example	$P \wedge Q$ is valid; $P \wedge \wedge Q$ is not	If $P=\text{True}$, $Q=\text{False}$, then $P \wedge Q = \text{False}$

SECTION 2: PROPOSITIONAL LOGIC - SYNTAX

2.1 Atomic Sentences (Propositions)

Atomic sentences are the simplest sentences in propositional logic. They consist of single propositional symbols (usually uppercase letters like P, Q, R, or meaningful names like "Raining", "Hot") that represent simple statements about the world. Each atomic sentence is indivisible - it cannot be broken down further.

Each proposition can take one of two **truth values**: TRUE (T, 1) or FALSE (F, 0). There is no middle ground - this is the **Law of Excluded Middle**. Additionally, a proposition cannot be both true and false at the same time - this is the **Law of Non-Contradiction**.

EXAMPLES OF ATOMIC SENTENCES

Using symbols: P, Q, R, S, A, B, C

Using meaningful names:

- "Raining" - It is raining
- "Hot" - The temperature is hot
- "EatingRice" - I am eating rice
- "TrafficJam" - There is a traffic jam

Special Constants:

- **True (⊤)**: A proposition that is always true in every model
- **False (⊥)**: A proposition that is always false in every model

2.2 Literals

A **literal** is either an atomic sentence (positive literal) or its negation (negative literal). Literals are important building blocks in many logical algorithms, especially in resolution-based theorem proving.

- **Positive Literal**: P, Q, Raining (the atom itself)
- **Negative Literal**: ¬P, ¬Q, ¬Raining (negation of the atom)

2.3 Complex Sentences using Logical Connectives

Complex sentences are formed by combining atomic sentences using **logical connectives**. These connectives allow us to express relationships between propositions. There are five primary connectives in propositional logic:

Connective	Symbol	Name	Read As	Example
1	¬	Negation (NOT)	not P	¬Raining = Not raining
2	∧	Conjunction (AND)	P and Q	Hot ∧ Sunny
3	∨	Disjunction (OR)	P or Q	Rain ∨ Snow
4	→	Implication (IF-THEN)	if P then Q	Rain → Wet
5	↔	Biconditional (IFF)	P if and only if Q	Pass ↔ Score≥50

SECTION 3: TRUTH TABLES AND SEMANTICS

3.1 Truth Tables for Logical Connectives

Truth tables define the semantics (meaning) of logical connectives by specifying the truth value of a complex sentence for every possible combination of truth values of its components. Let's examine each connective:

3.2 Negation (NOT) - $\neg P$

Negation simply flips the truth value. If P is true, $\neg P$ is false, and vice versa.

P	$\neg P$
T	F
F	T

3.3 Conjunction (AND) - $P \wedge Q$

Conjunction is true ONLY when BOTH operands are true. Think of it as a strict requirement - both conditions must be satisfied.

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

3.4 Disjunction (OR) - $P \vee Q$

Disjunction is true when AT LEAST ONE operand is true (inclusive OR). It's only false when both are false.

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

3.5 Implication (IF-THEN) - $P \rightarrow Q$

Implication is the most important and often misunderstood connective. $P \rightarrow Q$ means "if P then Q". It's FALSE only when P is true but Q is false (a broken promise). If P is false, the implication is vacuously true.

P	Q	$P \rightarrow Q$	Explanation
T	T	T	Promise kept
T	F	F	Promise broken!
F	T	T	No promise made, Q still true
F	F	T	No promise made (vacuously true)

■■ IMPORTANT: UNDERSTANDING IMPLICATION

The implication $P \rightarrow Q$ can be read in several equivalent ways:

- If P then Q
- P implies Q
- P only if Q
- Q if P
- P is sufficient for Q
- Q is necessary for P

Key insight: An implication $P \rightarrow Q$ is equivalent to $\neg P \vee Q$ (not P OR Q).

Why? The only way $P \rightarrow Q$ is false is when P is true and Q is false. In all other cases, it's true.

3.6 Biconditional (IFF) - $P \leftrightarrow Q$

Biconditional is true when both sides have the SAME truth value. It means "P if and only if Q" - both implications must hold: $P \rightarrow Q$ AND $Q \rightarrow P$.

P	Q	$P \leftrightarrow Q$	Explanation
T	T	T	Both true - same value
T	F	F	Different values
F	T	F	Different values
F	F	T	Both false - same value

SECTION 4: OPERATOR PRECEDENCE

Just like in arithmetic where multiplication is performed before addition, logical operators have a precedence order that determines how expressions are evaluated when parentheses are not used.

Priority	Operator	Name	Evaluation Order
1 (Highest)	$\neg$	Negation	First - applied to its operand
2	$\wedge$	Conjunction (AND)	Second
3	$\vee$	Disjunction (OR)	Third
4	$\rightarrow$	Implication	Fourth
5 (Lowest)	$\leftrightarrow$	Biconditional	Last

■ WORKED EXAMPLE: EVALUATING WITH PRECEDENCE

Expression: $\neg P \wedge Q \vee R \rightarrow S$

Step-by-step evaluation (following precedence):

- 1. First, apply $\neg$ to P: $(\neg P) \wedge Q \vee R \rightarrow S$
- 2. Next, evaluate $\wedge$: $((\neg P) \wedge Q) \vee R \rightarrow S$
- 3. Then, evaluate $\vee$: $((\neg P) \wedge Q) \vee R \rightarrow S$
- 4. Finally, evaluate $\rightarrow$: $((\neg P) \wedge Q) \vee R \rightarrow S$

Tip: When in doubt, use parentheses to make your intended meaning clear!

SECTION 5: IMPORTANT LOGICAL EQUIVALENCES

Two sentences are **logically equivalent** ($\equiv$) if they have the same truth value in every possible model. These equivalences are essential for simplifying logical expressions and proving theorems.

Name	Equivalence	Description
Double Negation	$\neg(\neg P) \equiv P$	Negating twice returns original
Commutative (AND)	$P \wedge Q \equiv Q \wedge P$	Order doesn't matter for AND
Commutative (OR)	$P \vee Q \equiv Q \vee P$	Order doesn't matter for OR
Associative (AND)	$(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$	Grouping doesn't matter for AND
Associative (OR)	$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$	Grouping doesn't matter for OR
Distributive	$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$	AND distributes over OR
Distributive	$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$	OR distributes over AND
De Morgan's Law	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$	NOT of AND = OR of NOTs
De Morgan's Law	$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$	NOT of OR = AND of NOTs
Implication	$P \rightarrow Q \equiv \neg P \vee Q$	IF-THEN as OR
Contrapositive	$P \rightarrow Q \equiv \neg Q \rightarrow \neg P$	Equivalent form of implication
Biconditional	$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$	IFF as two implications

■ DE MORGAN'S LAWS - MUST REMEMBER!

De Morgan's Laws are among the most important equivalences in logic:

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

"NOT (P AND Q)" is the same as "(NOT P) OR (NOT Q)"

Example: "It's not the case that it's hot AND sunny" = "It's not hot OR it's not sunny"

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

"NOT (P OR Q)" is the same as "(NOT P) AND (NOT Q)"

Example: "It's not the case that it's raining OR snowing" = "It's not raining AND it's not snowing"

SECTION 6: INFERENCE AND ENTAILMENT

6.1 What is Entailment?

Entailment (written as $\models$) is the fundamental concept connecting sentences in logic. We say that a knowledge base KB entails a sentence α (written $KB \models \alpha$) if α is true in every model where KB is true. In other words, if KB is true, then α MUST be true - there's no possible world where KB is true but α is false.

Example: If $KB = \{P, P \rightarrow Q\}$, then $KB \models Q$. Why? In any model where both P is true and $P \rightarrow Q$ is true, Q must also be true. There's no way to make P true and $P \rightarrow Q$ true while having Q be false.

6.2 Inference vs. Entailment

Concept	Symbol	Meaning	Nature
Entailment	$KB \models \alpha$	α is true in all models where KB is true	Semantic (about truth)
Inference	$KB \vdash \alpha$	α can be derived from KB using rules	Syntactic (about derivation)

6.3 Sound and Complete Inference

■ CRITICAL CONCEPTS: SOUNDNESS AND COMPLETENESS

SOUNDNESS: An inference procedure is **sound** if it only derives sentences that are actually entailed by the KB. In other words, it never makes things up - everything it concludes is actually true.

Formally: If $KB \vdash \alpha$ then $KB \models \alpha$ (if we can derive it, it's true)

COMPLETENESS: An inference procedure is **complete** if it can derive every sentence that is entailed by the KB. It doesn't miss anything - if something is true, it can find it.

Formally: If $KB \models \alpha$ then $KB \vdash \alpha$ (if it's true, we can derive it)

IDEAL: We want inference procedures that are BOTH sound AND complete:
 $KB \vdash \alpha$ if and only if $KB \models \alpha$

6.4 Important Inference Rules

Rule Name	Form	Description	Example
Modus Ponens	$P, P \rightarrow Q \vdash Q$	If P and P implies Q, then Q	Rain, $\text{Rain} \rightarrow \text{Wet} \vdash \text{Wet}$
Modus Tollens	$\neg Q, P \rightarrow Q \vdash \neg P$	If not Q and P implies Q, then not P	$\neg \text{Wet}, \text{Rain} \rightarrow \text{Wet} \vdash \neg \text{Rain}$
And-Elimination	$P \wedge Q \vdash P \text{ (or } Q)$	From conjunction, derive either part	$\text{Hot} \wedge \text{Sunny} \vdash \text{Hot}$
And-Introduction	$P, Q \vdash P \wedge Q$	From two facts, form conjunction	Hot, Sunny $\vdash \text{Hot} \wedge \text{Sunny}$
Or-Introduction	$P \vdash P \vee Q$	From one fact, form disjunction	Rain $\vdash \text{Rain} \vee \text{Snow}$

Resolution	$P \vee Q, \neg P \vee R \blacksquare Q \vee R$	Resolve complementary literals	$A \vee B, \neg A \vee C \blacksquare B \vee C$
■ MODUS PONENS - THE MOST IMPORTANT RULE Modus Ponens (Latin: "method of affirming") is the fundamental rule of logical inference: Structure: Premise 1: P (the antecedent is true) Premise 2: $P \rightarrow Q$ (the implication holds) Conclusion: Q (therefore the consequent is true) Real-world example: Premise 1: It is raining (P) Premise 2: If it is raining, then the ground is wet ($P \rightarrow Q$) Conclusion: Therefore, the ground is wet (Q) Why it works: Look at the truth table for implication. The only row where P is true AND $P \rightarrow Q$ is true is the row where Q is also true.			

SECTION 7: INTRODUCTION TO FIRST-ORDER LOGIC

7.1 Limitations of Propositional Logic

While propositional logic is simple and useful, it has significant limitations:

- Cannot represent objects and their properties individually
- Cannot express relationships between objects
- Cannot make general statements (e.g., 'All birds can fly')
- Each fact requires a separate proposition

7.2 First-Order Logic (FOL) Extensions

First-Order Logic (also called Predicate Logic) extends propositional logic with:

Element	Description	Examples
Constants	Specific objects in the domain	John, Mary, 5, Delhi
Variables	Placeholders for any object	x, y, z
Predicates	Properties or relations	Bird(x), Loves(x,y), GreaterThan(x,y)
Functions	Mappings from objects to objects	Father(x), Age(x), Plus(x,y)
Quantifiers	Express generality	$\forall$ (for all), $\exists$ (there exists)

7.3 Quantifiers

UNIVERSAL AND EXISTENTIAL QUANTIFIERS

Universal Quantifier ($\forall$): "For all" - makes statements about every object

$\forall x \text{ Bird}(x) \rightarrow \text{CanFly}(x)$ means "For all x, if x is a bird then x can fly"

Read as: "All birds can fly"

Existential Quantifier ($\exists$): "There exists" - asserts existence of at least one object

$\exists x \text{ Bird}(x) \wedge \neg \text{CanFly}(x)$ means "There exists an x such that x is a bird and x cannot fly"

Read as: "Some bird cannot fly" (like a penguin!)

Important Relationship:

$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ (Not all P = Some not P)

$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$ (No P exists = All are not P)

SECTION 8: SAMPLE QUIZ QUESTIONS

8.1 Multiple Choice Questions

1. What is the truth value of $P \rightarrow Q$ when P is FALSE?

- a) Always FALSE
- b) Always TRUE
- c) Depends on Q
- d) Undefined

Answer: b) Always TRUE (vacuously true when antecedent is false)

2. Which logical equivalence is De Morgan's Law?

- a) $P \wedge Q \equiv Q \wedge P$
- b) $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
- c) $P \rightarrow Q \equiv \neg P \vee Q$
- d) $\neg(\neg P) \equiv P$

Answer: b) $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$

3. In Modus Ponens, given P and $P \rightarrow Q$, we can conclude:

- a) P
- b) $\neg P$
- c) Q
- d) $\neg Q$

Answer: c) Q

4. Which operator has the HIGHEST precedence?

- a) $\wedge$ (AND)
- b) $\vee$ (OR)
- c) $\rightarrow$ (IMPLIES)
- d) $\neg$ (NOT)

Answer: d) $\neg$ (NOT)

5. $P \leftrightarrow Q$ is TRUE when:

- a) P and Q are both TRUE only
- b) P and Q have the same truth value
- c) At least one is TRUE
- d) P implies Q

Answer: b) P and Q have the same truth value

6. An inference procedure that only derives true conclusions is called:

- a) Complete
- b) Sound
- c) Valid
- d) Consistent

Answer: b) Sound

7. The symbol $\forall$ in first-order logic means:

- a) There exists
- b) For some
- c) For all
- d) Implies

Answer: c) For all (universal quantifier)

8. $P \rightarrow Q$ is logically equivalent to:

- a) $Q \rightarrow P$

b) $\neg P \rightarrow \neg Q$

c) $\neg Q \rightarrow \neg P$

d) $P \wedge Q$

Answer: c) $\neg Q \rightarrow \neg P$ (contrapositive)

8.2 True/False Questions

1. $P \wedge Q$ is true only when both P and Q are true. **(TRUE)**
2. The implication $P \rightarrow Q$ is false when P is false. **(FALSE (vacuously true when P is false))**
3. Modus Tollens allows us to conclude $\neg P$ from $\neg Q$ and $P \rightarrow Q$. **(TRUE)**
4. De Morgan's Law: $\neg(P \vee Q) \equiv \neg P \vee \neg Q$ **(FALSE (should be $\neg P \wedge \neg Q$))**
5. A sound inference procedure may derive false conclusions. **(FALSE (sound means only true conclusions))**
6. In propositional logic, we can express 'All birds can fly'. **(FALSE (need first-order logic))**
7. $\exists x$ means 'for all x'. **(FALSE ($\exists$ means 'there exists'))**
8. $P \leftrightarrow Q$ is equivalent to $(P \rightarrow Q) \wedge (Q \rightarrow P)$. **(TRUE)**

SECTION 9: KEY FORMULAS AND QUICK REFERENCE

ESSENTIAL FORMULAS TO REMEMBER

TRUTH TABLE SUMMARY:

- $\neg T = F, \neg F = T$
- $T \wedge T = T$, all else F
- $F \vee F = F$, all else T
- $T \rightarrow F = F$, all else T
- $P \leftrightarrow Q = T$ when P and Q have same value

KEY EQUIVALENCES:

- $P \rightarrow Q \equiv \neg P \vee Q$ (Implication as OR)
- $P \rightarrow Q \equiv \neg Q \rightarrow \neg P$ (Contrapositive)
- $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ (De Morgan)
- $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ (De Morgan)
- $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$

INFERENCE RULES:

- Modus Ponens: $P, P \rightarrow Q \vdash Q$
- Modus Tollens: $\neg Q, P \rightarrow Q \vdash \neg P$
- Resolution: $P \vee Q, \neg P \vee R \vdash Q \vee R$

OPERATOR PRECEDENCE (high to low):

$\neg \rightarrow \wedge \rightarrow \vee \rightarrow \rightarrow \rightarrow \leftrightarrow$

QUANTIFIER NEGATION:

- $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$
- $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$

— End of Revision Note 3 —
Best of luck with your exam preparation!